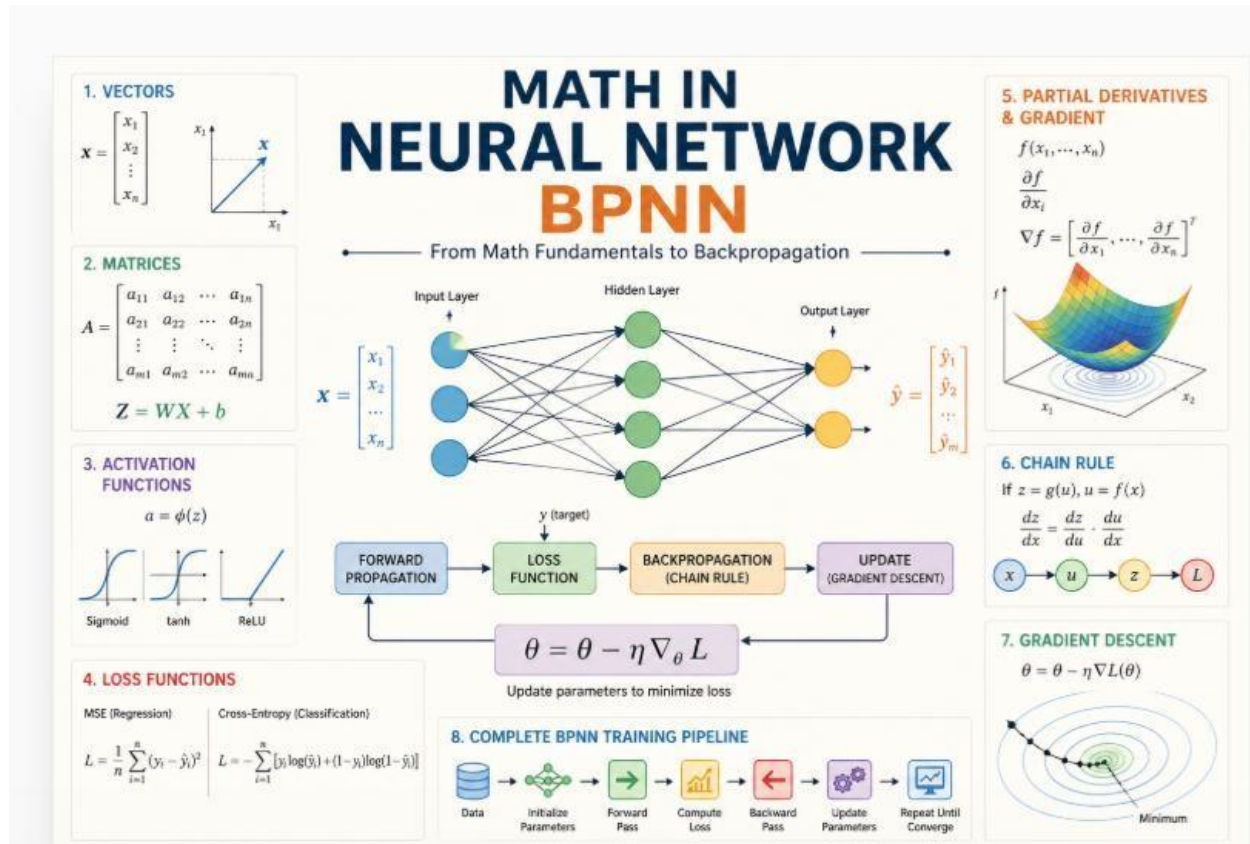


A Course on the Mathematical Foundations of Neural Networks



Prof Happy AI, 2026

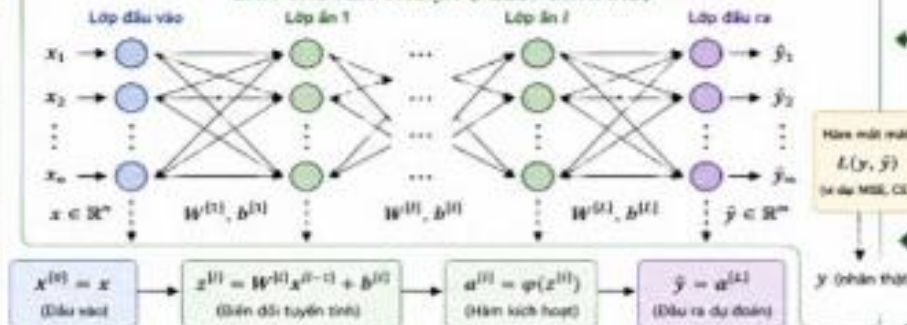
Mối quan hệ giữa các khái niệm toán học và mạng nơ-ron lan truyền ngược BPNN

CÁC KHÁI NIỆM TOÁN HỌC MỀN TẢNG

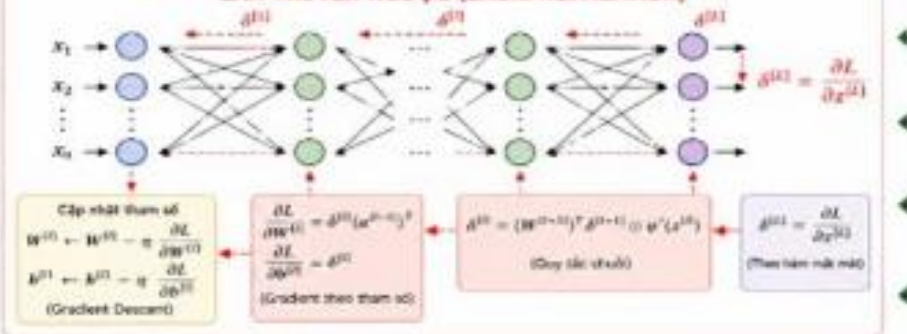
	1. Không gian vector $x \in \mathbb{R}^n$ Dữ liệu là vector
	2. Biến đổi tuyến tính (Affine) $z = Wx + b$ W : ma trận trọng số b : vector bias
	3. Hàm kích hoạt $a = \varphi(z)$ Đưa phi tuyến vào mô hình
	4. Hàm mất mát (Loss) $L(y, \hat{y})$ Đo lường sai số dự đoán
$\frac{\partial L}{\partial \theta}$	5. Đạo hàm riêng $\frac{\partial L}{\partial \theta}$ Độ nhạy của L theo tham số θ
	6. Quy tắc chuỗi $\frac{\partial L}{\partial \theta} = \frac{\partial L}{\partial a} \frac{\partial a}{\partial \theta}$ Lan truyền đạo hàm qua các lớp
	7. Tối ưu hóa (Gradient Descent) $\theta \leftarrow \theta - \eta \nabla_{\theta} L$ Cập nhật tham số để giảm thiểu
	8. Không gian hàm $f_{\theta} : \mathbb{R}^n \rightarrow \mathbb{R}^m$ Mạng nơ-ron là hàm phi tuyến tham số hóa

x : Đầu vào
 a : Tín hiệu hoạt động (activation)
 z : Tổng cộng (sum)
 y : Nhãn target (target)
 W, b : Tham số trọng số, bias
 L : Hàm mất mát
 i : Chỉ số lớp (1, 2, ..., L)

LAN TRUYỀN THUẬN (FEEDFORWARD)



LAN TRUYỀN NGƯỢC (BACKPROPAGATION)



TÓM TẮT QUY TRÌNH HỌC (TRAINING LOOP)



MỐI LIÊN HỆ GIỮA TOÁN HỌC VÀ BPNN

	1. Không gian vector Dữ liệu đầu vào là vector $x \in \mathbb{R}^n$
	2. Biến đổi tuyến tính Mỗi neuron thực hiện $z = Wx + b$ (Biến đổi tuyến tính)
	3. Hàm kích hoạt Áp dụng $\varphi(z)$ để thêm phi tuyến, giúp mô hình học quan hệ phức tạp
	4. Hàm mất mát Đo sai khác giữa dự đoán $\hat{y}$ và nhãn thật y
$\frac{\partial L}{\partial \theta}$	5. Đạo hàm riêng Tính độ nhạy của L theo từng tham số (W, b)
	6. Quy tắc chuỗi Lan truyền gradient từ đầu ra về đầu vào qua các lớp
	7. Tối ưu hóa Cập nhật W, b theo hướng giảm thiểu để hội tụ
	8. Không gian hàm Mạng nơ-ron tìm hàm f_{θ} tối ưu trong không gian hàm rất lớn

KÝ HIỆU
 $\odot$: Nhân từng phần tử (element-wise product)
 $\odot^T$: Tích tổng phần tử (inner product)
 $\varphi'(z)$: Đạo hàm của hàm kích hoạt φ
 L : Sai số (loss)
 i : Chỉ số lớp

Dưới đây là dự thảo Syllabus (Đề cương khóa học) được thiết kế chuyên sâu dành cho sinh viên CNTT hoặc các nhà nghiên cứu bắt đầu tiếp cận AI. Khóa học này kết nối chặt chẽ giữa lý thuyết toán học thuần túy và ứng dụng thực tế thông qua ngôn ngữ Python, lấy mô hình **Backpropagation Neural Network (BPNN)** làm mục tiêu cuối cùng.

Tên khóa học: Toán học cho Trí tuệ nhân tạo: Từ Cơ bản đến Thực hành BPNN

Thời lượng: 8 - 10 Buổi (hoặc chương)

Ngôn ngữ lập trình: Python (sử dụng NumPy, Matplotlib)

MỤC TIÊU KHÓA HỌC

- **Về kiến thức:** Nắm vững 8 khái niệm toán học nền tảng: Không gian vector, Biến đổi Affine, Hàm kích hoạt, Hàm mất mát, Đạo hàm riêng, Quy tắc chuỗi, Tối ưu hóa Gradient Descent và Không gian hàm.
- **Về kỹ năng:** Có khả năng tự tay lập trình (from scratch) một mạng nơ-ron lan truyền ngược mà không cần dùng đến các thư viện cấp cao như PyTorch hay TensorFlow.

NỘI DUNG CHI TIẾT

Chương 1: Không gian Vector và Dữ liệu số (Vector Spaces)

- **Toán học:** Định nghĩa vector trong $\mathbb{R}^n$, tích vô hướng (dot product), chuẩn (norm) của vector. Tại sao dữ liệu đầu vào (hình ảnh, văn bản) lại là các vector?
- **Lập trình Python:** Sử dụng `numpy.array`, các phép toán vector hóa (vectorization) để tối ưu hiệu năng thay vì dùng vòng lặp `for`.
- **Minh họa:** Biểu diễn một tấm ảnh grayscale thành một vector phẳng.

Chương 2: Ma trận và Biến đổi Tuyến tính (Affine Transformation)

- **Toán học:** Ma trận trọng số W , vector bias b . Phép biến đổi $z = Wx + b$. Ý nghĩa vật lý của việc xoay và co giãn không gian dữ liệu.
- **Lập trình Python:** Nhân ma trận với `np.dot` hoặc toán tử `@`. Khởi tạo trọng số ngẫu nhiên.
- **Minh họa:** Dùng ma trận để xoay hoặc dịch chuyển một tập hợp các điểm dữ liệu trên đồ thị 2D.

Chương 3: Hàm kích hoạt và Tính Phi tuyến (Activation Functions)

- **Toán học:** Tại sao cần phi tuyến? Giới thiệu các hàm Sigmoid, Tanh, ReLU. Áp dụng $\alpha = \phi(z)$.
- **Lập trình Python:** Định nghĩa các hàm toán học bằng NumPy. Vẽ đồ thị các hàm này để thấy sự khác biệt về miền giá trị.
- **Minh họa:** Giải thích tại sao nếu không có hàm kích hoạt, mạng nơ-ron sâu cũng chỉ là một phép biến đổi tuyến tính đơn giản.

Chương 4: Hàm mất mát và Đo lường Sai số (Loss Functions)

- **Toán học:** Khoảng cách giữa dự đoán $\hat{y}$ và thực tế y . Chi tiết về Mean Squared Error (MSE) và Cross-Entropy Loss.
- **Lập trình Python:** Viết hàm tính Loss.
- **Minh họa:** Vẽ bề mặt lỗi (Loss Surface) để thấy mục tiêu của việc huấn luyện là tìm "thung lũng" thấp nhất.

Chương 5: Đạo hàm riêng và Ý nghĩa của sự thay đổi (Partial Derivatives)

- **Toán học:** Đạo hàm riêng $\frac{\partial L}{\partial \theta}$. Khái niệm Gradient - vector chỉ hướng tăng nhanh nhất của hàm số.
- **Lập trình Python:** Tính đạo hàm bằng phương pháp sai phân số học (numerical differentiation) để kiểm chứng lý thuyết.
- **Minh họa:** Cực trị của hàm đa biến và cách gradient trở về phía cực đại.

Chương 6: Quy tắc chuỗi - Trái tim của Backpropagation (Chain Rule)

- **Toán học:** Đạo hàm của hàm hợp. Cách lan truyền sự thay đổi từ lớp đầu ra về lớp đầu vào thông qua các lớp ẩn: $\frac{\partial L}{\partial w} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w}$.
- **Lập trình Python:** Xây dựng cấu trúc tính toán theo từng bước (computational graph).

Chương 7: Tối ưu hóa với Gradient Descent

- **Toán học:** Công thức cập nhật trọng số: $\theta \leftarrow \theta - \eta \nabla_{\theta} L$. Tốc độ học (Learning Rate) η và sự hội tụ.
- **Lập trình Python:** Viết vòng lặp cập nhật trọng số. Thử nghiệm với các tốc độ học khác nhau (quá lớn gây phân kỳ, quá nhỏ gây chậm).

Chương 8: Tổng kết - Xây dựng mạng BPNN hoàn chỉnh

- **Lý thuyết:** Tổng hợp 8 bước thành một quy trình khép kín (Training Loop):
 1. Khởi tạo W, b .
 2. Lan truyền thuận (Feedforward): Tính $\hat{y}$ qua các lớp ẩn.
 3. Tính toán sai số (Loss).
 4. Lan truyền ngược (Backpropagation): Tính gradient cho từng tầng cảm biến.
 5. Cập nhật tham số (Gradient Descent).
- **Dự án cuối khóa:** Lập trình một lớp `NeuralNetwork` từ đầu để phân loại tập dữ liệu đơn giản (ví dụ: cổng Logic XOR hoặc bộ dữ liệu chữ viết tay MNIST cơ bản).
- **Thảo luận:** Không gian hàm và khả năng xấp xỉ vạn năng của mạng nơ-ron.

PHƯƠNG PHÁP GIẢNG DẠY

1. **Lý thuyết trực quan:** Luôn bắt đầu bằng hình ảnh minh họa (giống như sơ đồ bạn đã cung cấp) để sinh viên nắm được "bức tranh lớn".
2. **Code-along:** Giảng viên và sinh viên cùng viết code từng hàm toán học.

3. **Visual Debugging:** Sử dụng thư viện Matplotlib để vẽ lại quá trình Loss giảm dần qua từng epoch, giúp sinh viên "thấy" được toán học đang hoạt động.